Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Zurückkehren, warten oder weiterrechnen? (6 Punkte)

Eine Sortierstation soll alle 200 ms eine neue Messung bearbeiten.

1. Im kooperativen Programm wird `MessungStep()` aus der Hauptschleife aufgerufen. In einem RTOS-Projekt gibt es stattdessen den Thread `Messung(void *argument)`. Wo darf eine Endlosschleife stehen und wie kommt jeweils die nächste Aufgabe zum Zug? (2 Punkte)
2. Der RTOS-Thread ruft `osDelay(7)` auf. Geben Sie seine Zustandsfolge bis zum nächsten Lauf an. Welche Einheit hat die Sieben und warum garantiert das Ablaufen der Wartezeit noch keinen sofortigen Start? (2 Punkte)
3. Jemand ersetzt den Aufruf durch die übliche `HAL_Delay(7)`. Wird der Thread dadurch Blocked? Können höher priorisierte Threads und zugelassene Interrupts trotzdem ausgeführt werden? (2 Punkte)

Aufgabe 2: Drei endliche Arbeiten auf einem Kern (10 Punkte)

Für eine Paketwaage gelten feste Prioritäten und Präemption. Alle API-, Interrupt- und Wechselkosten sind null. Die Tabelle enthält Freigaben und eigene CPU-Zeiten, keine Boardmessungen.

Thread	Priorität	Freigaben / Verhalten	CPU-Zeit pro Job
Ausschleusen	höchste	wartet bis 3 ms; ein Job, danach wieder warten	2 ms
Wiegen	mittlere	periodisch bei 0, 6, 12 ms; nach jedem Job bis zur nächsten Freigabe warten	2 ms
Etikettbericht	niedrigste	ein Job bei 0; nach Abschluss warten	5 ms

Für Wiegen gilt die relative Deadline $D = 3\,\text{ms}$. Der Ausschleusethread ist bei null bereits Blocked; andere Arbeit fehlt. Leerlauf wird als Idle bezeichnet.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 15)\,\text{ms}$. Kennzeichnen Sie die Ursachen der Wechsel. (4 Punkte)

2. Nennen Sie die Zustände der drei Threads bei 2,5 und 3,5 ms. (2 Punkte)
3. Berechnen Sie die CPU-Zeitanteile einschließlich Idle in diesem Intervall und die Antwortzeiten aller drei Wiegejobs. (2 Punkte)
4. Ausschleusen erhält zusätzlich einen zweiten Job bei 6 ms. Wann startet und endet der ebenfalls bei 6 ms freigegebene Wiegejob? Hält er seine Deadline? (2 Punkte)

Aufgabe 3: Ein Tickplan mit und ohne Drift (8 Punkte)

1. Ein Kernel hat $f_{\text{tick}} = 250\,\text{Hz}$. Berechnen Sie Tickperiode und aufgerundete Tickzahlen für 6 ms und 18 ms. Warum garantiert `osDelay(1)` keine physische Mindestwartezeit von einer ganzen Tickperiode? (3 Punkte)
2. Für diese Teilaufgabe gilt stattdessen ein 1-kHz-Tick. Drei Arbeitsphasen benötigen nacheinander 3, 1 und 5 ms. Der erste Start ist bei null, alle Arbeitsenden liegen genau auf Tickgrenzen; weitere Störungen fehlen. Bestimmen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(12)`“ und bei einem absoluten 12-Tick-Sollraster. (3 Punkte)
3. Eine Arbeit endet bei Tick 53; der nächste geplante Start bei Tick 48 ist bereits verpasst. Die Periode beträgt 12 Ticks. Geben Sie eine geeignete Reaktion ohne sofortige Nachholserie an. Was kann ein Zähler verpasster Warteziele aussagen, was ohne zusätzliche Deadlinedefinition nicht? (2 Punkte)

Aufgabe 4: Ressourcen und falsche Abhilfen (8 Punkte)

1. Drei CMSIS-Threads erhalten Stacks mit 768, 1536 und 640 Byte. Pro Thread werden zusätzlich 96 Byte Verwaltungsdaten angenommen. Berechnen Sie die Summe. Welche weiteren Anteile fehlen für eine vollständige Speicherbilanz? Wie viele Byte sind 192 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)
2. Ein Diagnose-Thread steht über einer Ausgabesteuerung, bleibt ständig Ready und ruft in seiner Rechenschleife `osThreadYield()` auf. Die Ausgabesteuerung bleibt ebenfalls Ready. Wird sie dadurch ausgewählt? Benennen Sie das Problem und eine geeignete Änderung. Es gibt keine gemeinsam gehaltene Ressource. (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe ist 6; vier Prioritätsbits und passende Gruppierung sind vorausgesetzt. Darf ein IRQ mit logischer Priorität 4 `osThreadFlagsSet` aufrufen? Darf ein hierfür zulässiger IRQ mit Priorität 7 `osThreadFlagsWait` aufrufen? (2 Punkte)

Aufgabe 5: Wie viele Hinweise bleiben erhalten?

Ein Thread wartet auf Bit 2, also die Maske `0x04U`. Ein zulässiger anderer Thread setzt dieses Bit bei 2, 3 und 9 ms. Der Empfänger ruft `osThreadFlagsWait(0x04U, osFlagsWaitAny, osWaitForever)` bei 5 und 11 ms und direkt nach dem zweiten erfolgreichen Wait noch einmal auf. Vorher ist das Bit gelöscht, weitere Set-Aufrufe fehlen. Erfolgreiche Waits löschen das angenommene Bit automatisch.

1. Welche beiden Waits kehren sofort zurück und was passiert beim dritten?
2. Wie viele Set-Aufrufe und wie viele erfolgreiche Waits gibt es? Warum sind diese Zahlen verschieden?
3. Wie erkennen Sie einen Fehler im Rückgabewert einer Flagfunktion? Weshalb wäre ein Vergleich mit `osOK` falsch?

Aufgabe 6: Eine Füllstandsüberwachung selbst implementieren

Programmieren, etwa 30–40 Minuten. Nutzen Sie die vorbereitete **NUCLEO-H755ZI-Q / STM32H755ZIT6**-Basis mit STM32CubeMX und VS Code mit STM32 VS Code Extension: FreeRTOS/CMSIS-RTOS2 auf CM4, Präemption, 32-Bit-Ticks bei 1000 Hz; CM7 mit generiertem Systemstart. Die Projektvorbereitung erfolgt vor dem Termin.

Schreiben Sie zwei Threadfunktionen: Ein **Mess-Thread** prüft simulierte Füllstände; ein höher priorisierter **Signal-Thread** meldet Grenzübertritte mit einem LED-Puls. `lab03_tank.c/.h` aus `code/lab03/` erzeugt beide Threads und stellt diese Hilfen bereit:

Schnittstelle / Vorgabe	Bedeutung
`Lab03_TankRead()`	nächster Wert aus 25, 45, 85, 90, 55, 30 Prozent, zyklisch
`Lab03_TankLed(on)`	nur vom Signal-Thread verwendete LED setzen
`Lab03_TankWaitNext(&next)`	nächstes absolutes 200-Tick-Ziel abwarten; bei verpasstem Ziel auf jetzt plus 200 ausrichten
`lab03_tank_alarm_handle`	gültiges Handle des Signal-Threads
`LAB03_TANK_FLAG`	Hinweisbit für einen neuen Grenzübertritt
`Lab03_TankFail()`	Fehlerpfad, kehrt nicht zurück

1. Ersetzen Sie in `tank_threads.c` die anfänglichen Wartefunktionen und entfernen Sie `WaitForImplementation`. Implementieren Sie `void Lab03_TankSampler(void *argument)`: sofort eine Probe, danach alle 200 Ticks eine neue. Setzen Sie mit `osThreadFlagsSet` das Hinweisbit nur beim Übergang von **unter 80** zu **mindestens 80 Prozent**; anfangs gilt „unter der Grenze“. Initialisieren Sie `next` vor der Schleife mit `osKernelGetTickCount()` und nutzen Sie die Wartehilfe am Schleifenende.
2. Implementieren Sie `void Lab03_TankAlarm(void *argument)`: mit `osThreadFlagsWait` blockierend auf den Hinweis warten, LED einschalten, `osDelay(80U)`, LED ausschalten; wiederholen. Prüfen Sie Flagfehler und Delay-Status.
3. Erhöhen Sie `lab03_tank_requests` vor jedem Set und `lab03_tank_responses` nach jedem erfolgreichen Wait. Jeder Zähler hat genau einen schreibenden Thread. Bei welchen Messwerten erwarten Sie einen Puls, weshalb bei 90 Prozent keinen weiteren?
4. Bauen, laden und prüfen Sie Ihre Implementierung: wiederkehrender kurzer Puls, fortschreitende Messfolge und passende Zählerstände. Lesen Sie die Diagnosewerte bei angehaltenem CM4; dieser Halt ist keine Zeitmessung.
5. Erklären Sie die Zustände beider Threads zwischen den Aktionen. Weshalb darf der höher priorisierte Signal-Thread eine Endlosschleife haben, ohne den Mess-Thread auszuhungern? Was würde ein wartendes `while` auf ein Flag daran ändern?

Erweiterung: Ersetzen Sie die Wartehilfe vorübergehend durch `osDelay(200U)`. Vergleichen Sie beide Zeitpläne bei variabler Arbeitszeit auf Papier. Stellen Sie das absolute Raster wieder her.

Ergebnis: eigene Threadfunktionen und Funktionsprüfung; ohne Boardlauf dessen Status angeben. Ein Flag meldet einen ausstehenden Hinweis, keinen verlustfreien Ereigniszähler.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Arm CMSIS-RTOS2; ST UM2408 und RM0399. Papierzeiten und Speichergrößen sind Lehrannahmen.